

Ansible File & Folder Management Modules

A comprehensive guide to managing files, directories, templates, and configurations across your infrastructure using Ansible's powerful file manipulation modules.



The File Module: Foundation of File Management

The file module serves as the cornerstone for managing files, directories, and symbolic links across your infrastructure. It provides comprehensive control over ownership, permissions, and filesystem objects.

Core Capabilities

- Create or delete files, directories, and symlinks
- Manage ownership with owner and group
- Set permissions using mode: (e.g., '0644', '0755')
- Apply changes recursively with recurse: yes

Use the state: parameter to specify the desired object type: file, directory, link, or absent for removal.



Archive Management: Compression & Extraction

Archive Module

Compresses files and directories on managed hosts into formats including zip, tar, and gz.

- `path`: defines files to compress
- `exclude_path`: skips specific items

Unarchive Module

Extracts compressed archives to specified destinations on remote hosts.

- `src`: specifies the archive file
- `dest`: sets extraction directory
- `remote_src`: true for archives already on remote



Content Assembly & File Operations

1

Assemble Module

Merges multiple file fragments into a single file, joining them in alphabetical order. Perfect for building configuration files from modular components.

- `src`: directory containing fragments
- `dest`: output file path
- `regexp`: filter matching files only

2

Copy Module

Copies files from control node to remote hosts, or between remote directories with `remote_src: yes`.

- Supports inline creation using `content`:
- `backup: true` creates timestamped backups
- Cannot copy from remote back to control node

3

Fetch Module

The reverse of copy—retrieves files from remote hosts to the control node, organising them by hostname.

- Creates subdirectories per host
- Prevents overwriting from different systems
- Ideal for collecting logs and configurations

Stat Module: Intelligent File Inspection



The `stat` module retrieves detailed information about files and directories, enabling conditional logic based on filesystem state.

Key Features

Use `register:` to capture results and access properties including:

- `isdir` – checks if path is a directory
- `islink` – identifies symbolic links
- `mode` – retrieves permission settings
- `exists` – confirms file presence

Commonly used in conditionals: run tasks only if files exist, are directories, or meet specific criteria.

Line-Level Configuration Management



Lineinfile Module

Adds, removes, or updates a single line within configuration files with surgical precision.

- `line:` defines the text content
- `path:` specifies target file
- `state: absent` removes the line
- `insertafter:` / `insertbefore:` control placement
- `regexp:` enables dynamic updates in loops

Use `check_mode: true` to validate changes without modifying files.



Blockinfile Module

Inserts or updates multiple lines as a cohesive text block within configuration files.

- `block:` defines multiline content using `|`
- Automatically wraps text with BEGIN/END markers
- `insertafter:` / `insertbefore:` define placement
- `create: true` creates file if missing

Ideal for managing configuration sections that span multiple lines.

Common Configuration Parameters

Most file-related modules share a consistent set of parameters for comprehensive control over filesystem attributes and security contexts.



Ownership & Permissions

`owner`, `group`, and `mode` control access rights and filesystem security across all file operations.



SELinux Context

`selevel`, `serole`, `setype`, and `seuser` define security context components for enhanced protection.



Safety Features

`backup`: creates timestamped backups, whilst `validate`: runs verification commands before applying changes.



Advanced Options

`attributes`: sets filesystem attributes (requires `chattr`), `unsafe_writes`: controls atomic operations, `create`: generates missing files.

Templates: Dynamic Configuration with Jinja2

Templates leverage the Jinja2 templating language to create dynamic, customisable configuration files that adapt to your infrastructure needs.

01

Template Creation

Store template files (e.g., `conf.properties.j2`) in the `templates/` directory with Jinja2 placeholders.

02

Variable Substitution

Placeholders like `{{ variables }}` are replaced with values from Ansible facts or custom variables at runtime.

03

Template Module

The template module renders the file and deploys it to managed hosts with full control over ownership, permissions, and backup options.

04

Security Best Practices

Secure sensitive values using Ansible Vault or secret management tools. Support for conditionals and file parameters ensures flexibility.

Advanced Jinja2 Features

Flow Statements

Use `{% for %}` loops and conditionals to iterate over lists and control rendering logic. Supports `{% else %}` for handling empty collections.

Macros

Define reusable template fragments with parameters and optional defaults, similar to functions. Use `|e` to escape special characters safely.

Template Inclusion

`{% include 'file.j2' %}` inserts another template into the current one, enabling modular template design and reuse.

Template Inheritance

`{% block name %}{% endblock %}` defines sections in base templates, which child templates override using `{% extends "base.j2" %}`.

Filters & Lookup Plugins

Jinja2 Filters

Transform and manipulate variable values using pipe (|) syntax for elegant data processing.

- |escape – sanitises special characters
- |capitalize – formats text capitalisation
- |pprint – pretty-prints data structures
- |join('-') – concatenates list items

Filters can be chained together for complex, multi-stage transformations.

Lookup Plugins

Retrieve external data dynamically during template rendering using lookup functions.

Example: `lookup('file', 'path/to/file')` reads file contents directly into your template.

Lookup plugins can access various data sources including files, environment variables, DNS records, and external APIs.

